



```
def action():  
    return "MOVIE MAGIC"
```

```
production = "AI_FILMMAKER"  
take = 1
```



PRÁCTICA CINEASTA IA

Construyendo una Productora de Cine Autónoma con Python

Objetivo: Transformar un PDF en un Tráiler de Cine usando Modelos Locales.

EL OBJETIVO: TU "ESTUDIO VIRTUAL" EN UNA CAJA



El Reto

- **Coherencia Visual:** Mantener el estilo sin usar servidores externos.
- **Hardware:** Optimizado para GPU de consumo (ej. NVIDIA T4/3060).
- **Coste Cero:** Utilización exclusiva de modelos Open Source.

La Solución

Crearemos una '**Tubería**' (**Pipeline**) que **lee, imagina, narra y edita automáticamente**.

EL EQUIPO DE PRODUCCIÓN (TECH STACK)



El Cerebro

**TinyLlama-
1.1B-Chat**

Analiza el texto y extrae la acción principal en inglés. Pequeño y rápido.



El Pintor

**Realistic
Vision V5.1**

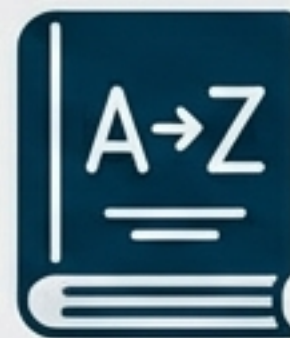
Versión de Stable Diffusion para fotografía cinematográficas y rostros reales.



El Locutor

Edge-TTS

Voces neuronales de Azure para la narración.



El Traductor

**Deep-
Translator**

Puente entre el pensamiento en inglés y la narración en español.



El Editor

MoviePy

Corta, pega y sincroniza audio con imagen.

CONFIGURACIÓN DEL SET (ENVIRONMENT SETUP)

Lógica

- **Instalación:** Librerías críticas para visión artificial y audio.
- **Detección de Hardware:** El script decide si usa GPU (rápido) o CPU (lento).

Producer Note:

¡Ojo! Usar `torch.float16` en GPU ahorra muchísima VRAM.

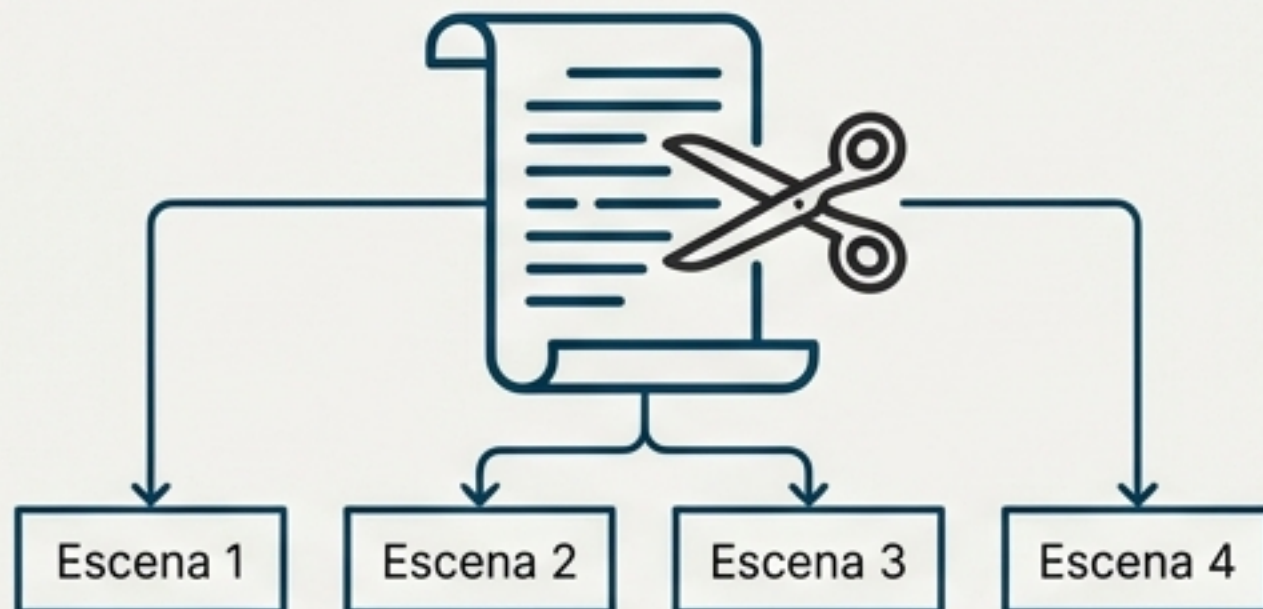
```
import torch
from diffusers import StableDiffusionPipeline
from moviepy.editor import ...

if torch.cuda.is_available():
    device = 'cuda'
    dtype = torch.float16 # Memoria eficiente
else:
    device = 'cpu'
```

FASE 1: EL GUIONISTA (INGESTA Y SEGMENTACIÓN)

El Concepto

- **El Problema:** La IA no puede leer un libro entero de golpe.
- **La Solución ("Chunking"):** Dividir el texto en bloques lógicos.
- **Estrategia:** Dividimos el total de frases en 4 bloques para generar 4 escenas.



```
def dividir_en_escenas(texto_completo):  
    # Limpieza básica  
    texto_limpio =  
    texto_completo.replace('\n', ' ')  
  
    # Split por puntos  
    frases = [f.strip() + '.' for f in  
    texto_limpio.split('.') if len(f) > 10]  
  
    # Lógica de división (Math)  
    paso = len(frases) // 4  
    chunks = []  
    # ... append chunks logic ...  
  
    return chunks[:4]
```

FASE 2: ANÁLISIS DE ESCENA (EL CEREBRO)

Prompt Engineering

- Usamos TinyLlama para resumir la acción.
- ¿Por qué en inglés?
Stable Diffusion entiende mejor el inglés.
- Prompt Structure:
Usamos el formato `<|user|>` y `<|assistant|>`.

```
def generar_prompt_base(texto_fragmento):  
  
    # Estructura Llama Chat  
    prompt = f'''<|user|>  
    Describe the main character actions in this text for a movie scene.  
    Text: "{texto_fragmento[:200]}"  
    <|assistant|>  
  
    ...  
  
    outputs = pipe(prompt, max_new_tokens=80, ...)
```

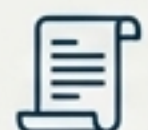
Brevedad para evitar alucinaciones.

DIRECCIÓN DE ARTE: INYECCIÓN DE CONTEXTO

Cómo evitar que la IA alucine imágenes genéricas.

Lógica Condicional

Si el LLM lee "villano",
generará un villano cualquiera.
Debemos 'Inyectar' código
duro para forzar la estética.



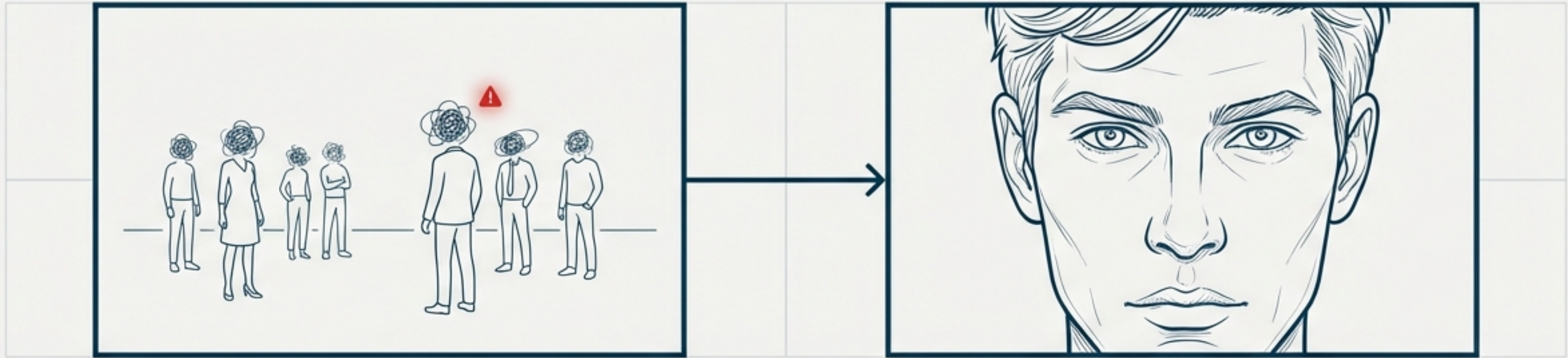
Ejemplo: Si detecta
'Doom', inyectamos
'chrome mask, green
hood'.

```
def inyectar_vestuario(texto_escena, prompt_base):  
    texto_lower = texto_escena.lower()  
  
    if 'doom' in texto_lower:  
        prompt_mejorado += ', (Doctor Doom villain  
        portrait:1.5), (wearing chrome metal mask...)'  
  
    return prompt_mejorado
```



Peso 1.5: Aumenta la
importancia de esta
palabra.

EL TRUCO DE CÁMARA: LA ESTRATEGIA "CLOSE-UP"



Wide Shot (Riesgo alto de deformación)

Close-Up (Alta fidelidad)

El Problema: Stable Diffusion 1.5 sufre con caras lejanas.

La **Solución**: Forzamos siempre un primer plano en el prompt.

```
prompt_mejorado = "movie poster close-up shot, " + prompt_base
```

🎬 **Producer Note:** Pedir 'detailed eyes' y 'perfect face structure' es obligatorio en esta fase.

FASE 3: EL ARTISTA (GENERACIÓN VISUAL)

Parámetros Clave

- **Modelo:**
Realistic_Vision_V5.1_noVAE
(Optimizado para realismo).
- **Negative Prompt:** Lista negra
(deformed, bad anatomy, cartoon).
- **Guidance Scale (7.0):** Balance entre
creatividad y fidelidad.
- **Steps (40):** Más pasos = piel más
limpia.

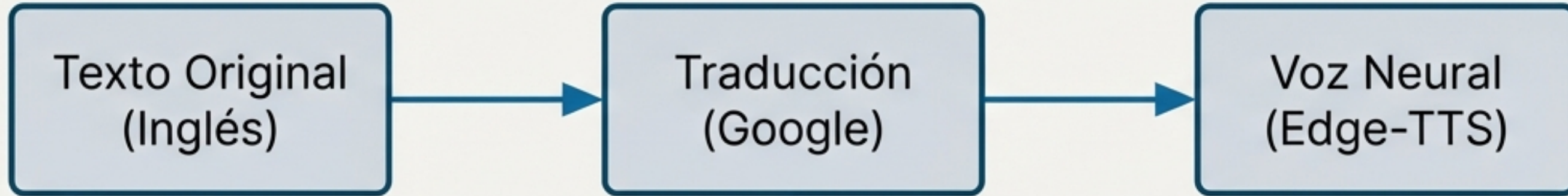
```
pipe(  
    prompt=prompt_final,  
    negative_prompt='deformed, bad anatomy, ...',  
    num_inference_steps=40,  
    guidance_scale=7.0,  
    height=512,  
    width=512  
)
```

Calidad

Calidad

Creatividad

FASE 3: EL LOCUTOR (AUDIO Y DOBLAJE)



```
async def generar_audio(texto_ingles, nombre_archivo):  
    # Paso 1: Traducir  
    traductor = GoogleTranslator(source='auto', target='es')  
    texto_espanol = traductor.translate(texto_ingles)  
  
    # Paso 2: Generar Voz  
    communicate = edge_tts.Communicate(texto_espanol, 'es-ES-AlvaroNeural')  
    await communicate.save(nombre_archivo)
```

 **Key Insight:** Usamos funciones asíncronas (**async/await**) para la generación de audio.

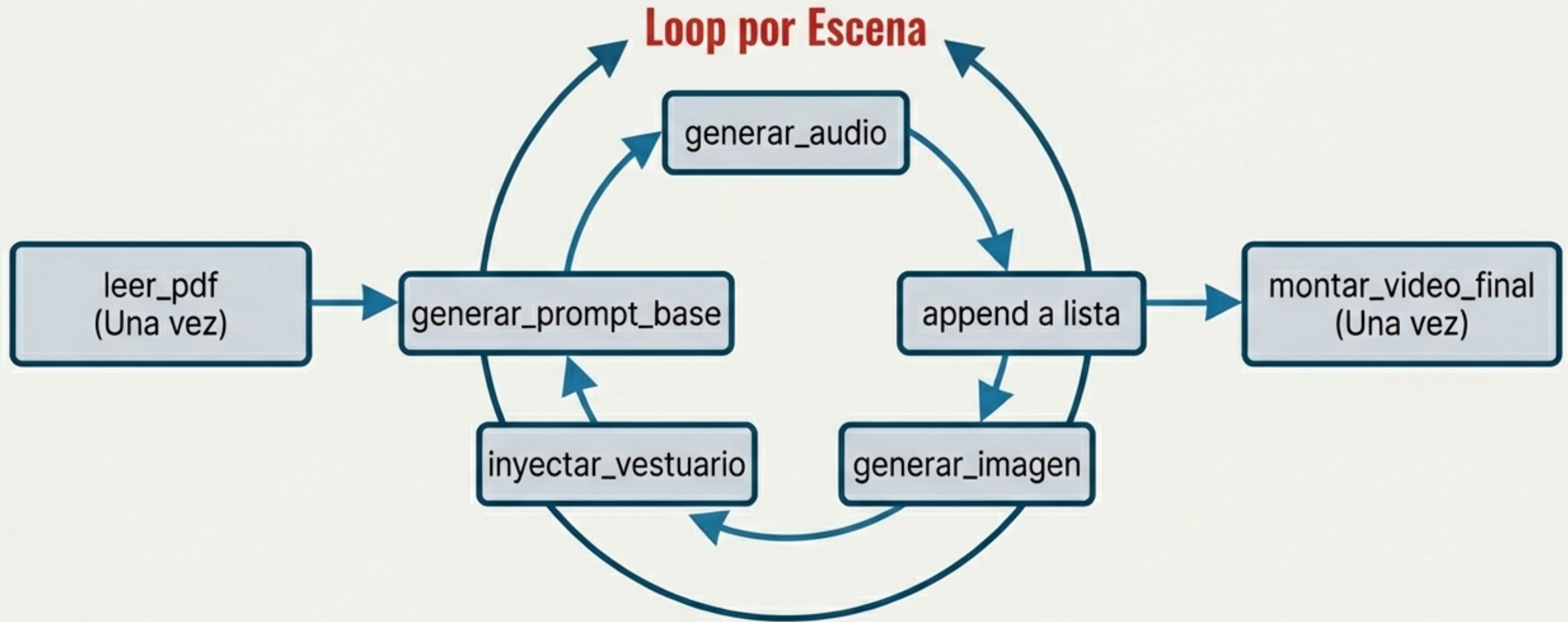
FASE 4: SALA DE MONTAJE (THE EDITOR)

Lógica de Edición

- Sincronización: Duración Imagen = Duración Audio.
- Transiciones: ``crossfadein(0.5)`` para suavizar el corte.
- Render: Compresión h264 para compatibilidad universal.

```
def montar_video_final(lista_escenas, salida):  
    clips = []  
    for escena in lista_escenas:  
        # Cargar Audio  
        audio = AudioFileClip(escena['audio'])  
  
        # Crear Clip de Imagen con duración del audio  
        clip = ImageClip(escena['img']).set_duration(audio.duration).set_audio(audio)  
  
        # Efecto de transición  
        clip = clip.crossfadein(0.5)  
        clips.append(clip)  
  
    final = concatenate_videoclips(clips, method='compose')
```

LA ORQUESTACIÓN: EL BUCLE PRINCIPAL



 **Producer Note:** El bucle itera sobre la lista `escenas_texto`. Si tienes 4 chunks, el bucle corre 4 veces.

SOLUCIÓN DE PROBLEMAS (DEBUGGING)

	Problema	Solución
1	Caras deformes o monstruosas.	Aumentar <code>num_inference_steps</code> o revisar que <code>'close-up'</code> esté en el prompt.
2	El personaje no se parece (ej. Doom sin máscara).	Revisar la función <code>inyectar_vestuario</code> . ¿Está bien escrita la palabra clave?
3	Audio desincronizado.	Verificar el bucle en MoviePy: <code>set_duration(audio.duration)</code> .
4	Error de memoria (CUDA OOM).	Ejecutar <code>flush()</code> y <code>gc.collect()</code> entre escenas.

NIVEL EXPERTO: RETOS DE PRODUCCIÓN



Cambio de Franquicia

Adapta el script para Star Wars o Harry Potter. Ej:

```
if 'vader' in text:  
    add 'black helmet,  
        lightsaber'
```



Cambio de Estilo

Sustituye Realistic Vision por un modelo Anime ('AnythingV5') o Disney ('ToonYou').

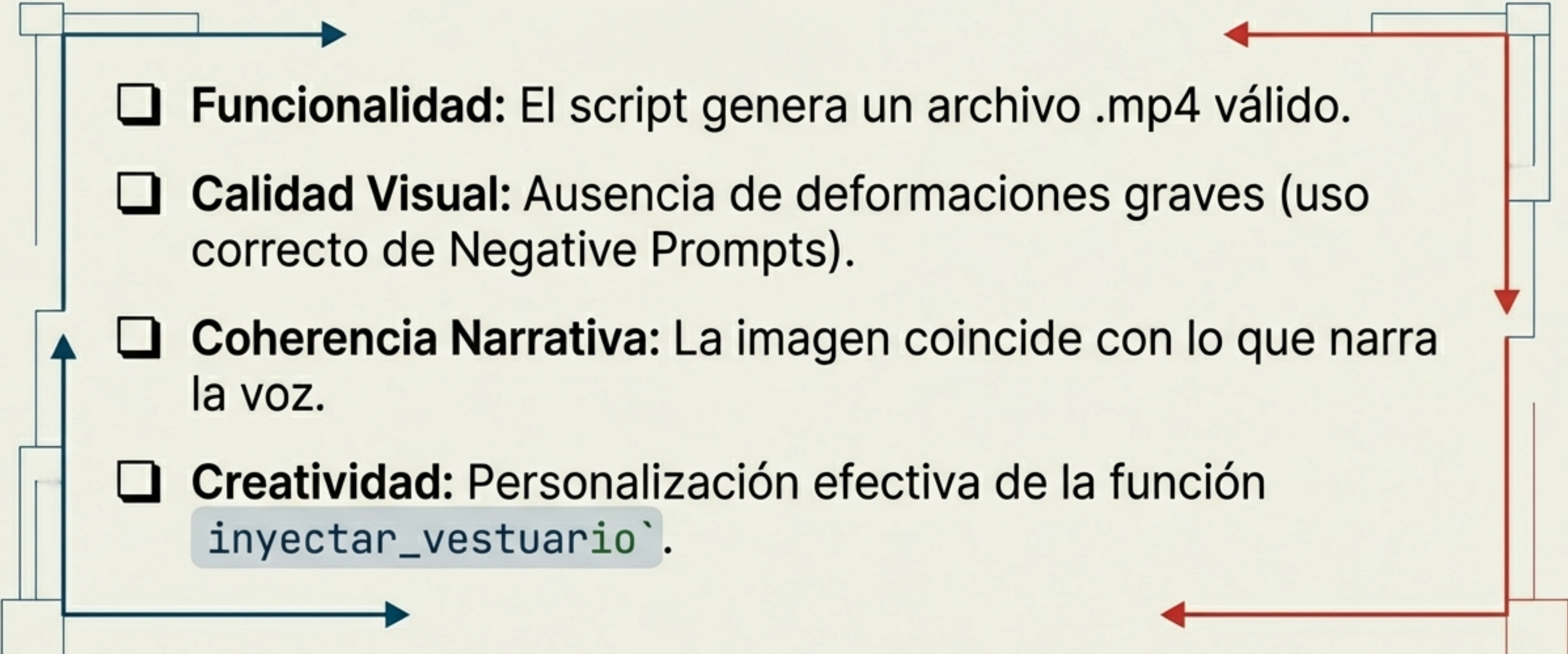


Banda Sonora

Añade música de fondo usando

`CompositeAudioClip` en MoviePy (bajar volumen para no tapar la voz).

EL ESTRENO: CRITERIOS DE EVALUACIÓN

- 
- ❑ **Funcionalidad:** El script genera un archivo .mp4 válido.
 - ❑ **Calidad Visual:** Ausencia de deformaciones graves (uso correcto de Negative Prompts).
 - ❑ **Coherencia Narrativa:** La imagen coincide con lo que narra la voz.
 - ❑ **Creatividad:** Personalización efectiva de la función `inyectar_vestuario``.

"Luces, Cámara, Código."